

Lab 5
Kernel Module

TA: Yu-Wen Huang

Prof.: Hsung-Pin Chang

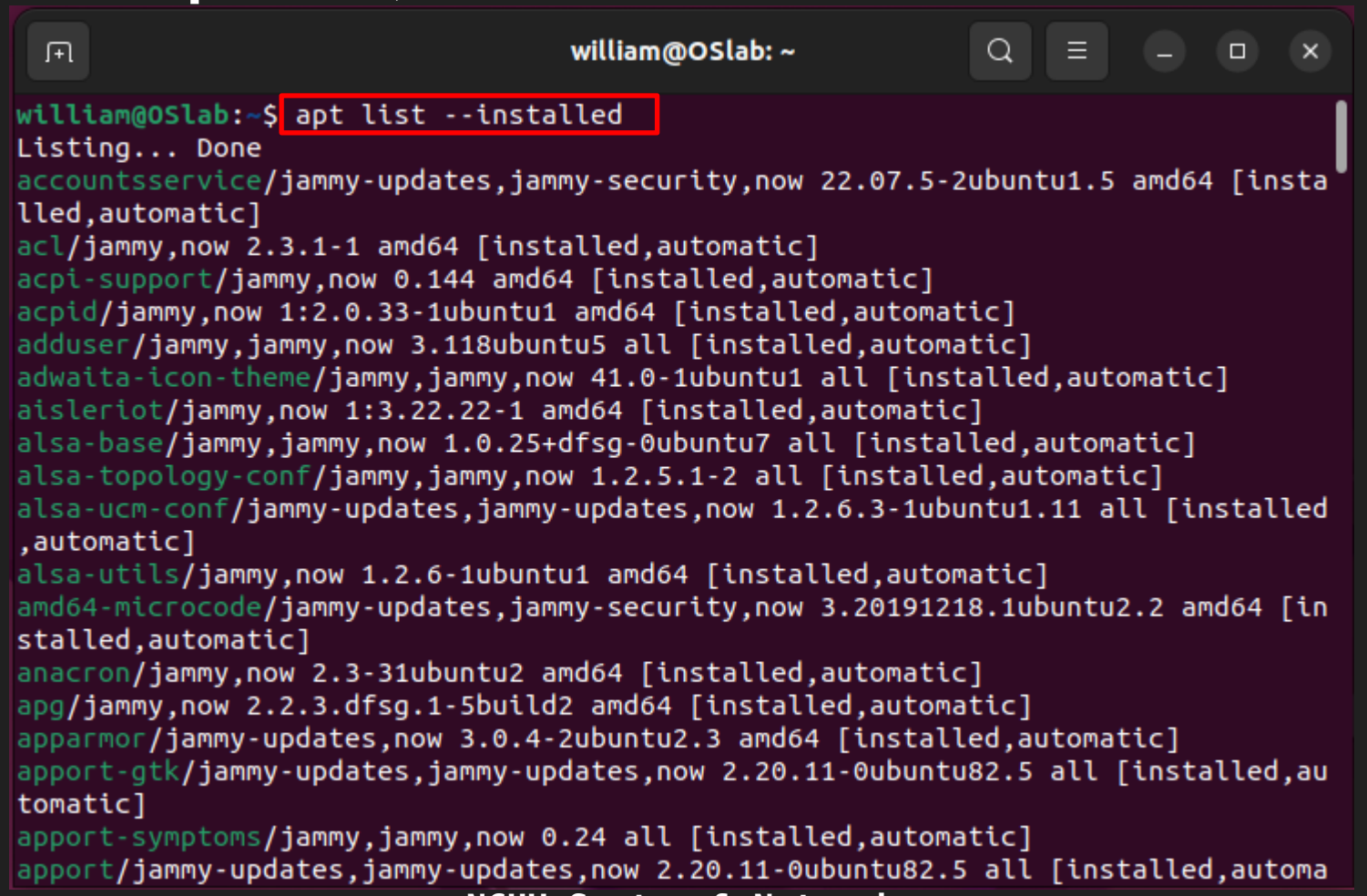
NCHU System & Network Lab

$ Outline

- Introduction
- Create a Kernel Module
- Module Parameters
- Symbols in Kernel Module
- Exercise

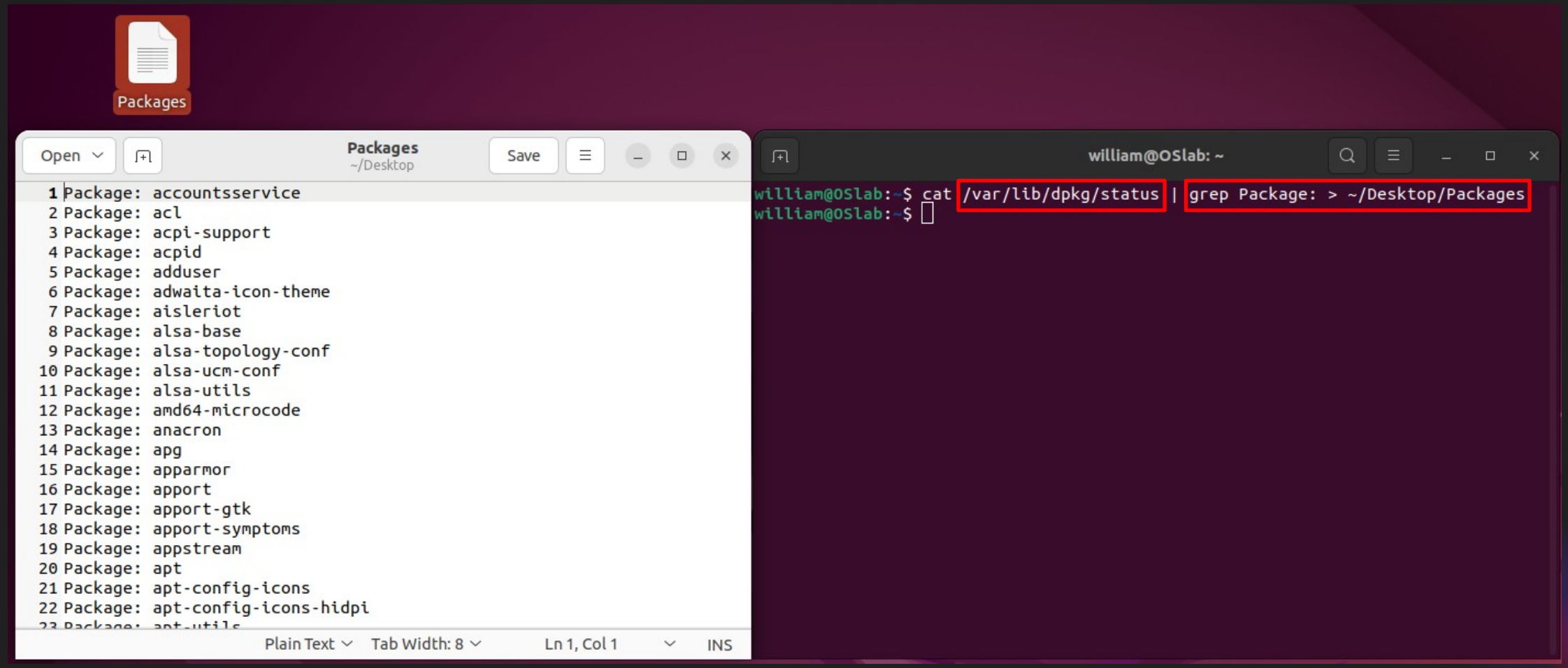
$ Introduction (1/4)

- 我們熟悉的 **apt** 指令，除了 **install** 之外還有什麼功能？



$ Introduction (2/4)

- 數數看已經安裝了幾個 **package**

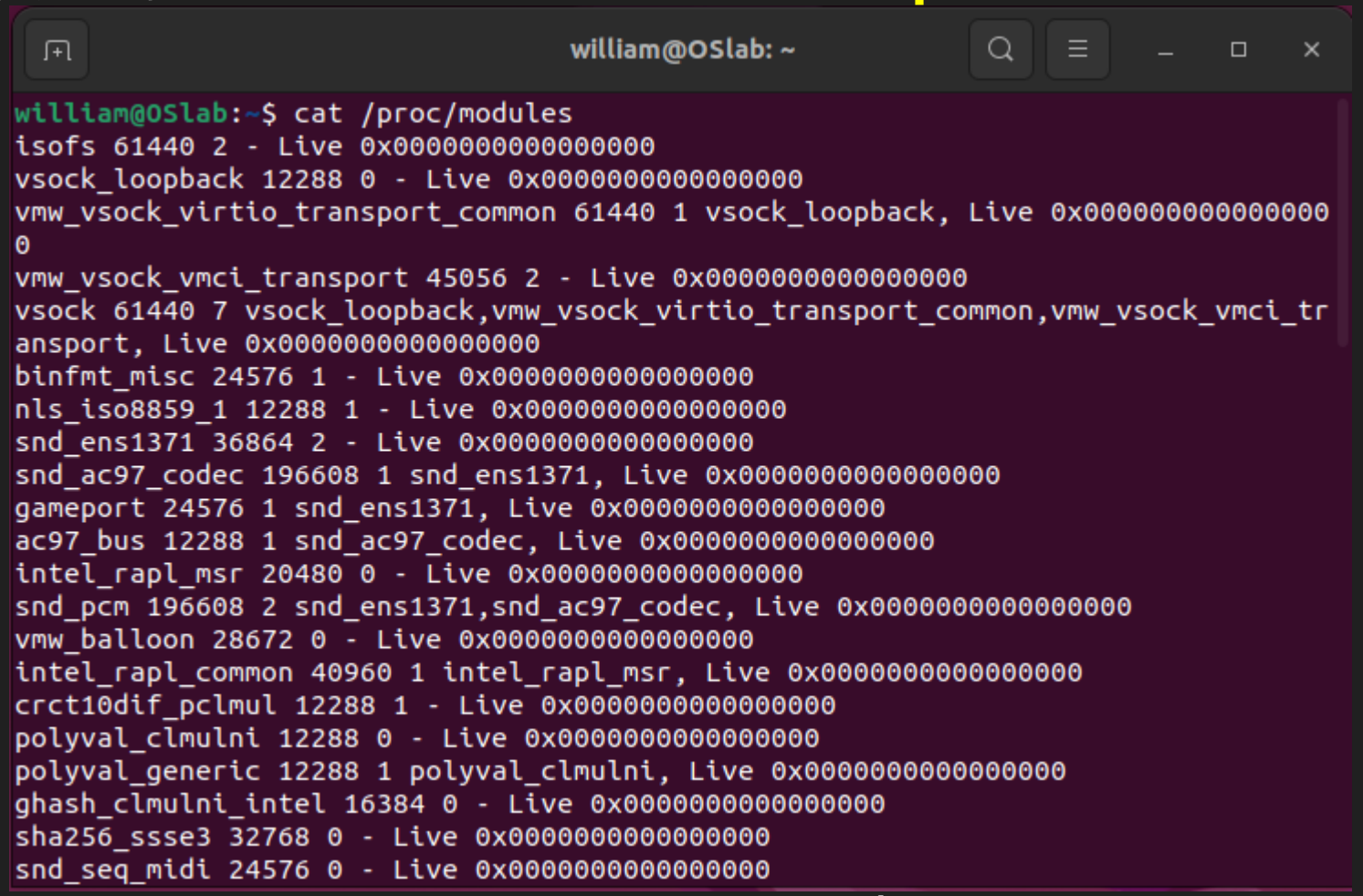


$ Introduction (3/4)

- **Package**
  - 存在於 **user space**，因此執行特定動作需要呼叫 **system call**
  - 例如 **apt, gcc, make...**
- **Kernel module**
  - 在運行中的 **Linux Kernel** 中動態載入和卸載的程式碼
  - 根據需求擴展 **Kernel**，以支援新的硬體設備、檔案系統等功能
  - 最常見的例子就是驅動程式

$ Introduction (4/4)

- 列出已安裝的 **kernel module: cat /proc/modules**



$ Outline

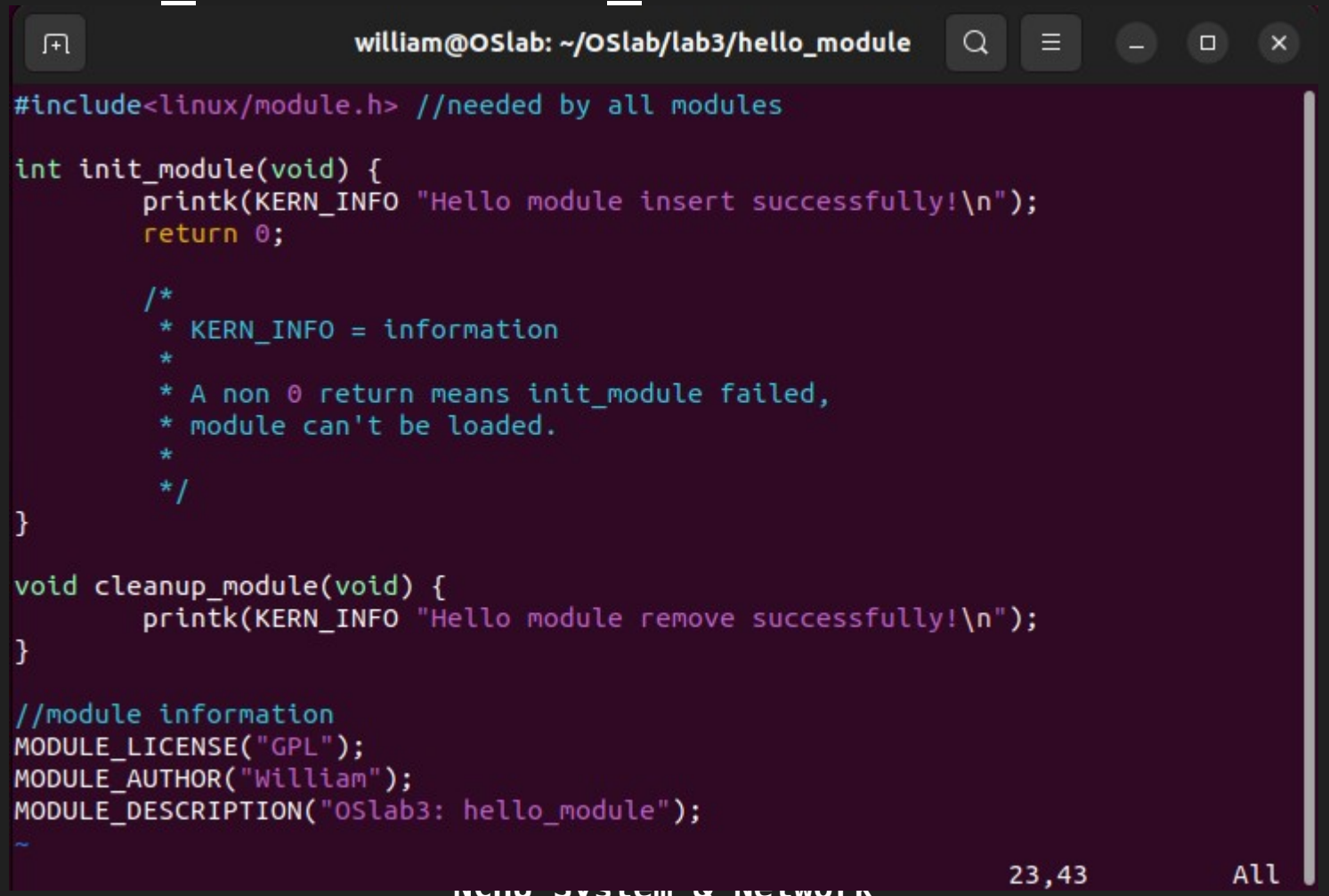
- Introduction
- Create a Kernel Module
- Module Parameters
- Symbols in Kernel Module
- Exercise

$ Create a Kernel Module (1/8)

- lab3 (資料夾)
  - hello_module (資料夾)
    - hello_module.c
    - Makefile

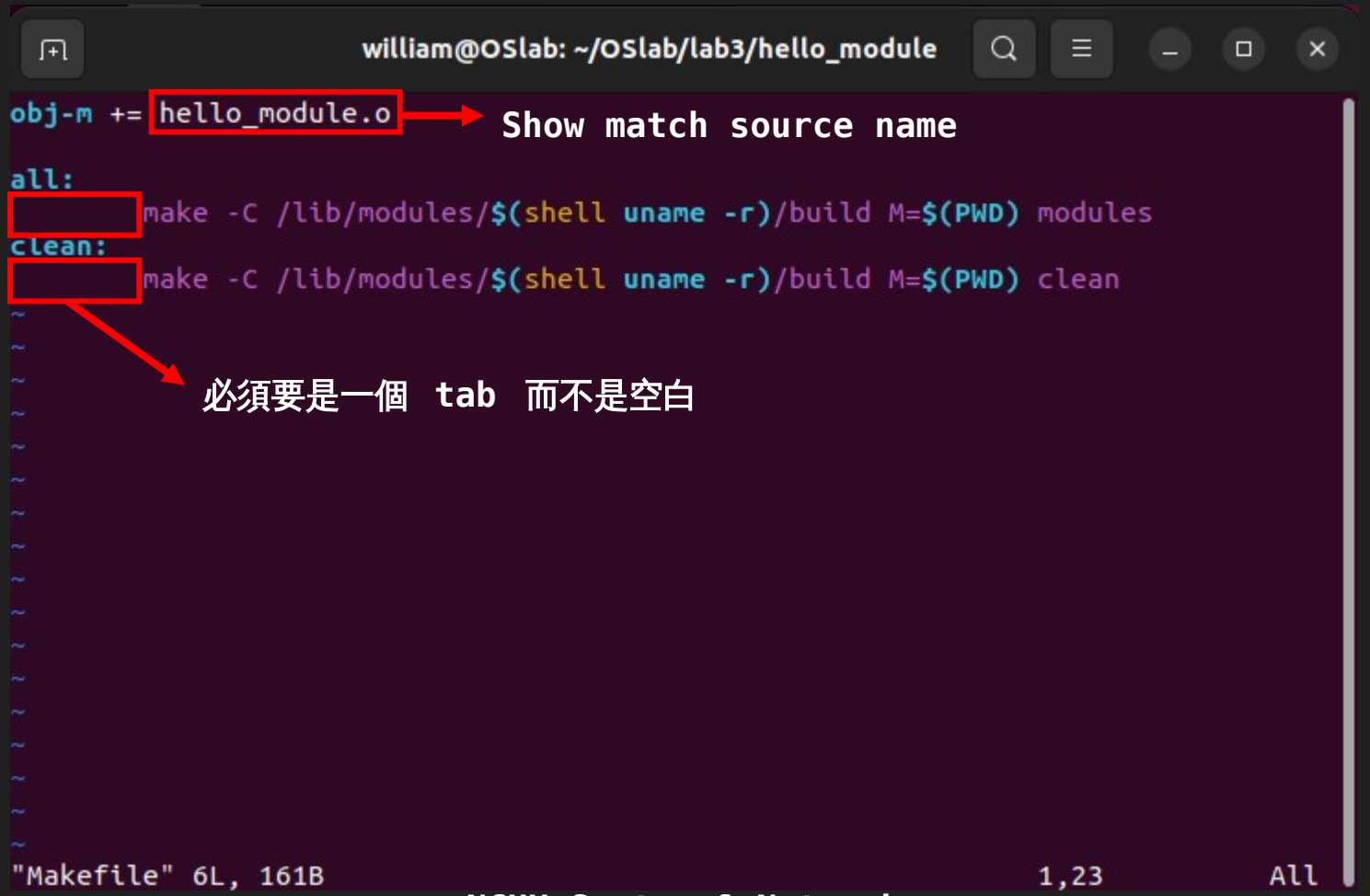
$ Create a Kernel Module (2/8)

- lab3/hello_module/hello_module.c



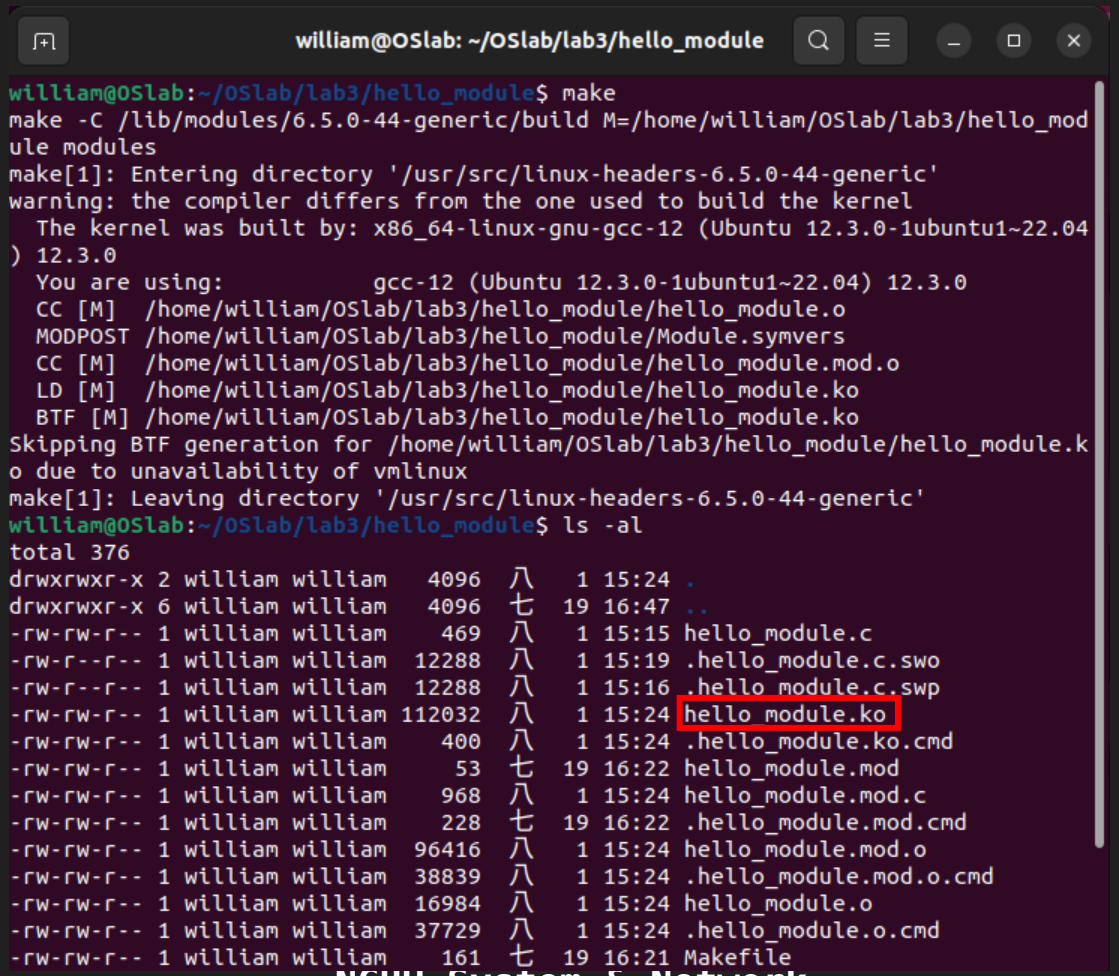
$ Create a Kernel Module (3/8)

- lab3/hello_module/Makefile



$ Create a Kernel Module (4/8)

- 終端機 **cd** 到 **hello_module** 資料夾，輸入 **make** 編譯成 **.ko** 檔



$ Create a Kernel Module (5/8)

- 查看 **module** 資訊：**modinfo hello_module.ko**

```
william@OSlab:~/OSlab/lab3/hello_module$ modinfo hello_module.ko
filename:       /home/william/OSlab/lab3/hello_module/hello_module.ko
description:    OSlab3: hello_module
author:         William
license:        GPL
srcversion:     8AFAC631E28B472EC87194C
depends:
retpoline:      Y
name:           hello_module
vermagic:       6.5.0-44-generic SMP preempt mod_unload modversions
```

$ Create a Kernel Module (6/8)

- 安裝 **kernel module**：**sudo insmod hello_module.ko**
- 使用 **lsmod** 指令查看 **hello_module** 是否成功安裝
  - **lsmod** 做的事情就是解析 **/proc/modules** 檔案並輸出

```
william@OSlab:~/OSlab/lab3/hello_module$ sudo insmod hello_module.ko
[sudo] password for william:
william@OSlab:~/OSlab/lab3/hello_module$ lsmod
Module                  Size  Used by
hello_module           12288  0
isofs                  61440  2
vsock_loopback         12288  0
vmw_vsock_virtio_transport_common    61440  1 vsock_loopback
vmw_vsock_vmci_transport    45056  2
vsock                  61440  7 vmw_vsock_virtio_transport_common,vsock_loopback
,vmw_vsock_vmci_transport
binfmt_misc            24576  1
nls_iso8859_1          12288  1
snd_ens1371            36864  2
snd_ac97_codec        196608  1 snd_ens1371
gameport               24576  1 snd_ens1371
```

$ Create a Kernel Module (7/8)

- 確認 **hello_module** 是否成功印出訊息：**sudo dmesg | tail**
  - **dmesg**：顯示 **kernel** 訊息
  - **tail**：顯示末 **10** 筆資料

```
william@OSlab:~/OSlab/lab3/hello_module$ sudo dmesg | tail
[100423.470286] audit: type=1400 audit(1722499678.929:232): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.configure
" pid=14072 comm="apparmor_parser"
[100423.679407] audit: type=1400 audit(1722499679.137:233): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.connect-p
lug-host-hunspell" pid=14073 comm="apparmor_parser"
[100423.890400] audit: type=1400 audit(1722499679.349:234): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.disconnec
t-plug-host-hunspell" pid=14074 comm="apparmor_parser"
[100424.107148] audit: type=1400 audit(1722499679.565:235): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.post-refr
esh" pid=14075 comm="apparmor_parser"
[100424.833284] audit: type=1400 audit(1722499680.293:236): apparmor="DENIED" op
eration="open" class="file" profile="snap-update-ns.firefox" name="/usr/local/sh
are/" pid=14098 comm="5" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
[109913.250800] e1000: ens33 NIC Link is Down
[109925.346650] e1000: ens33 NIC Link is Up 1000 Mbps Full Duplex, Flow Control:
 None
[114694.508661] hello_module: loading out-of-tree module taints kernel.
[114694.508689] hello_module: module verification failed: signature and/or requi
red key missing - tainting kernel
[114694.522666] Hello module insert successfully!
```

$ Create a Kernel Module (8/8)

- 解除安裝 kernel module：**sudo rmmod hello_module**



因為已經成為 module，因此後面不用加 .ko

```
william@OSlab:~/OSlab/lab3/hello_module$ sudo rmmod hello_module
william@OSlab:~/OSlab/lab3/hello_module$ sudo dmesg | tail
[100423.679407] audit: type=1400 audit(1722499679.137:233): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.connect-p
lug-host-hunspell" pid=14073 comm="apparmor_parser"
[100423.890400] audit: type=1400 audit(1722499679.349:234): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.disconnec
t-plug-host-hunspell" pid=14074 comm="apparmor_parser"
[100424.107148] audit: type=1400 audit(1722499679.565:235): apparmor="STATUS" op
eration="profile_replace" profile="unconfined" name="snap.firefox.hook.post-refr
esh" pid=14075 comm="apparmor_parser"
[100424.833284] audit: type=1400 audit(1722499680.293:236): apparmor="DENIED" op
eration="open" class="file" profile="snap-update-ns.firefox" name="/usr/local/sh
are/" pid=14098 comm="5" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
[109913.250800] e1000: ens33 NIC Link is Down
[109925.346650] e1000: ens33 NIC Link is Up 1000 Mbps Full Duplex, Flow Control:
 None
[114694.508661] hello_module: loading out-of-tree module taints kernel.
[114694.508689] hello_module: module verification failed: signature and/or requi
red key missing - tainting kernel
[114694.522666] Hello module insert successfully!
[115061.597396] Hello module remove successfully!
```

$ Outline

- Introduction
- Create a Kernel Module
- Module Parameters
- Symbols in Kernel Module
- Exercise

$ Module Parameters (1/5)

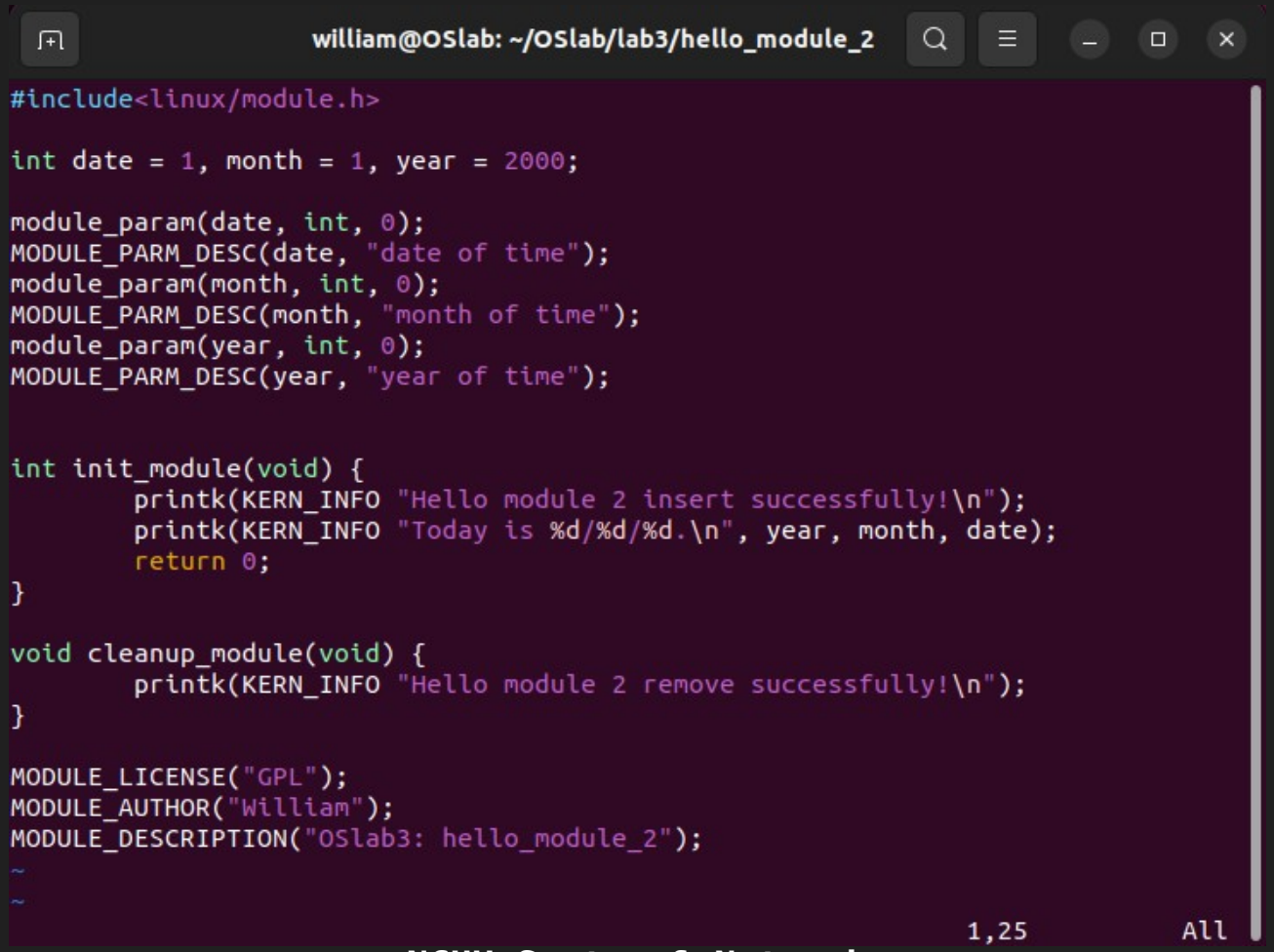
- Two macros about parameters
  - **module_param(name, type, param)**
    - Declare a parameter
    - name: variable name
    - type: variable type
    - param: not discuss here, set to 0
  - **MODULE_PARM_DESC(name, desc)**
    - Provide a parameter information for users
    - desc: variable description

$ Module Parameters (2/5)

- lab3 (資料夾)
  - hello_module (資料夾)
    - hello_module.c
    - Makefile
  - **hello_module_2 (資料夾)**
    - **hello_module_2.c**
    - **Makefile**

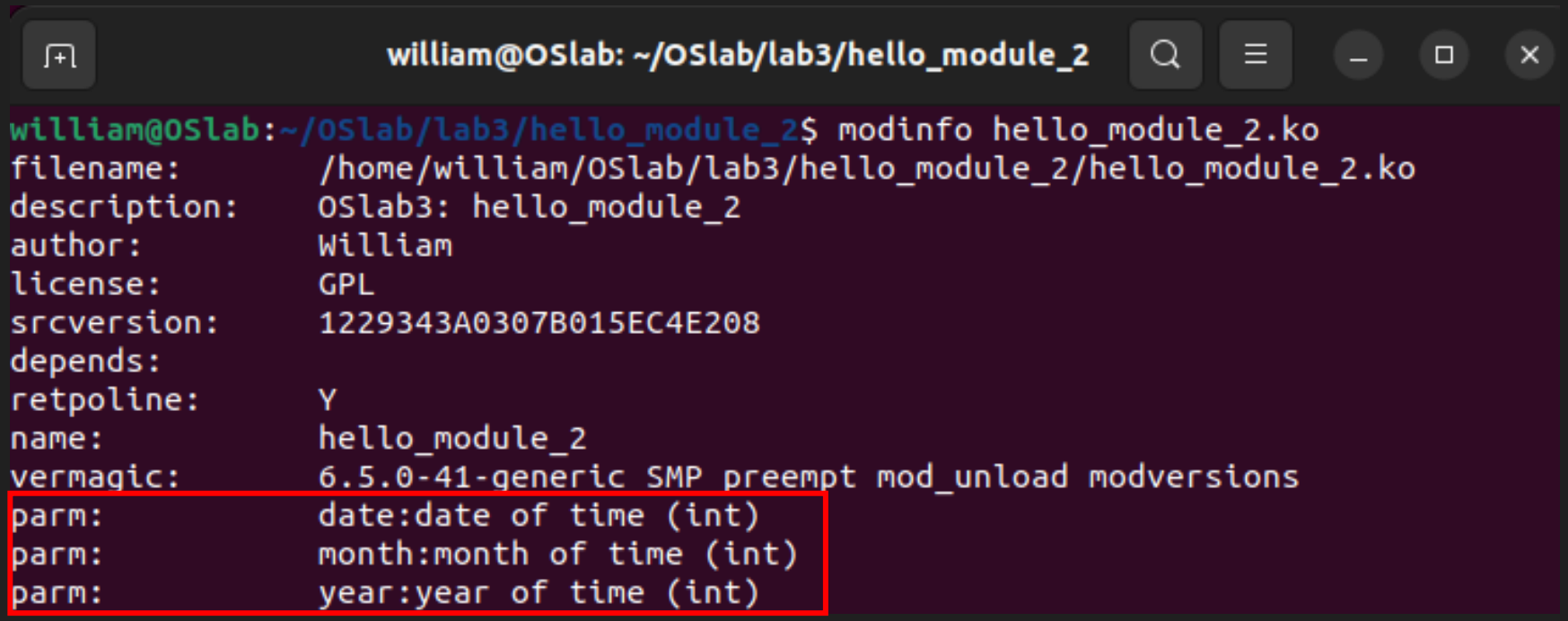
$ Module Parameters (3/5)

- lab3/hello_module_2/hello_module_2.c



$ Module Parameters (4/5)

- MODULE_PARM_DESC 會反映在 modinfo 上



$ Module Parameters (5/5)

```
william@OSlab:~/OSlab/lab3/hello_module_2$ sudo insmod hello_module_2.ko year=20
24 month=8 date=1
[sudo] password for william:
william@OSlab:~/OSlab/lab3/hello_module_2$ sudo dmesg | tail -3
[116619.291001] hello_module_2: disagrees about version of symbol module_layout
[116728.806376] Hello module 2 insert successfully!
[116728.806386] Today is 2024/8/1.
```

$ Outline

- Introduction
- Create a Kernel Module
- Module Parameters
- Symbols in Kernel Module
- Exercise

$ Symbols in Kernel Module (1/7)

- 在剛才的例子中，我們所宣告的變數和函式都是 **symbol** 的一種
  - **int year = 2023; -> variable symbol**
  - **void doSomething() { ... } -> function symbol**
- **EXPORT_SYMBOL(symbol_name) macro** 使得其它模組可以使用它們

$ Symbols in Kernel Module (2/7)

- lab3 (資料夾)
  - hello_module (資料夾)
  - hello_module_2 (資料夾)
  - **symbol (資料夾)**
    - **symbol_A.c**
    - **symbol_B.c**
    - **Makefile**

$ Symbols in Kernel Module (3/7)

- lab3/symbol/symbol_A.c

william@OSlab: ~/OSlab/lab3/symbol

```
#include<linux/module.h>

int symbol_variable = 99;       // variable symbol
int symbol_function(int x){     // function symbol
        if (x > 0) return 1;
        else if (x == 0) return 0;
        else return -1;
}

int init_module(void) {
        printk(KERN_INFO "Symbol A insert successfully!\n");
        return 0;
}

void cleanup_module(void) {
        printk(KERN_INFO "Symbol A remove successfully!\n");
}

EXPORT_SYMBOL(symbol_variable);
EXPORT_SYMBOL(symbol_function);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("William");
MODULE_DESCRIPTION("OSlab3: symbol_A");
~
                                                    2,0-1         All
```

$ Symbols in Kernel Module (4/7)

- lab3/symbol/symbol_B.c

william@OSlab: ~/OSlab/lab3/symbol

```
#include<linux/module.h>

extern int symbol_variable;
extern int symbol_function(int);

int init_module(void) {
        printk(KERN_INFO "Symbol B insert successfully!\n");

        printk(KERN_INFO "Symbol_variable: %d\n", symbol_variable);
        printk(KERN_INFO "Symbol_function(100): %d\n", symbol_function(100));
        printk(KERN_INFO "Symbol_function(0): %d\n", symbol_function(0));
        printk(KERN_INFO "Symbol_function(-100): %d\n", symbol_function(-100));

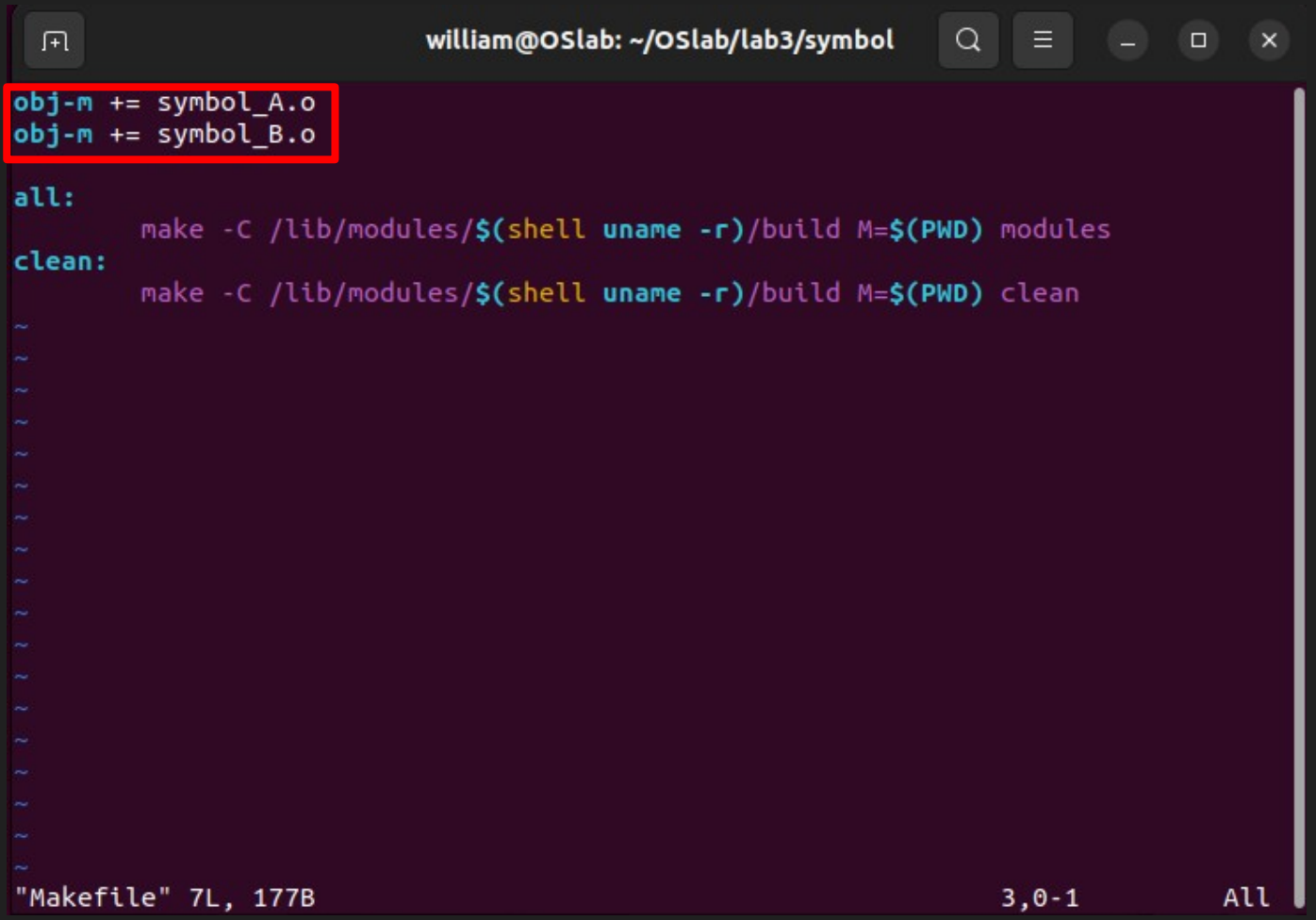
        return 0;
}

void cleanup_module(void) {
        printk(KERN_INFO "Symbol B remove successfully!\n");
}

MODULE_LICENSE("GPL");
MODULE_AUTHOR("William");
MODULE_DESCRIPTION("OSlab3: symbol_A");
~
~
"symbol_B.c" 23L, 631B                                2,0-1         All
```

$ Symbols in Kernel Module (5/7)

- lab3/symbol/Makefile



$ Symbols in Kernel Module (6/7)

- 因為 **B** 使用 **A** 的 **symbol**，因此需要先安裝 **symbol_A**

```
william@OSlab:~/OSlab/lab3/symbol$ sudo insmod symbol_A.ko
[sudo] password for william:
william@OSlab:~/OSlab/lab3/symbol$ sudo insmod symbol_B.ko
william@OSlab:~/OSlab/lab3/symbol$ lsmod | head -3
Module                  Size  Used by
symbol_B               12288  0
symbol_A               12288  1 symbol_B
william@OSlab:~/OSlab/lab3/symbol$ sudo dmesg | tail
[116619.291001] hello_module_2: disagrees about version of symbol module_layout
[116728.806376] Hello module 2 insert successfully!
[116728.806386] Today is 2024/8/1.
[117313.261925] symbol_A: disagrees about version of symbol module_layout
[117366.294677] Symbol A insert successfully!
[117379.369673] Symbol B insert successfully!
[117379.369683] Symbol_variable: 99
[117379.369690] Symbol_function(100): 1
[117379.369696] Symbol_function(0): 0
[117379.369702] Symbol_function(-100): -1
```

$ Symbols in Kernel Module (7/7)

- 相反地，需要先解除安裝 **symbol_B**，才能解除安裝 **symbol_A**

```
william@OSlab:~/OSlab/lab3/symbol$ sudo rmmod symbol_A
rmmod: ERROR: Module symbol_A is in use by: symbol_B
```

$ Outline

- Introduction
- Create a Kernel Module
- Module Parameters
- Symbols in Kernel Module
- Exercise

$ Exercise (1/5)

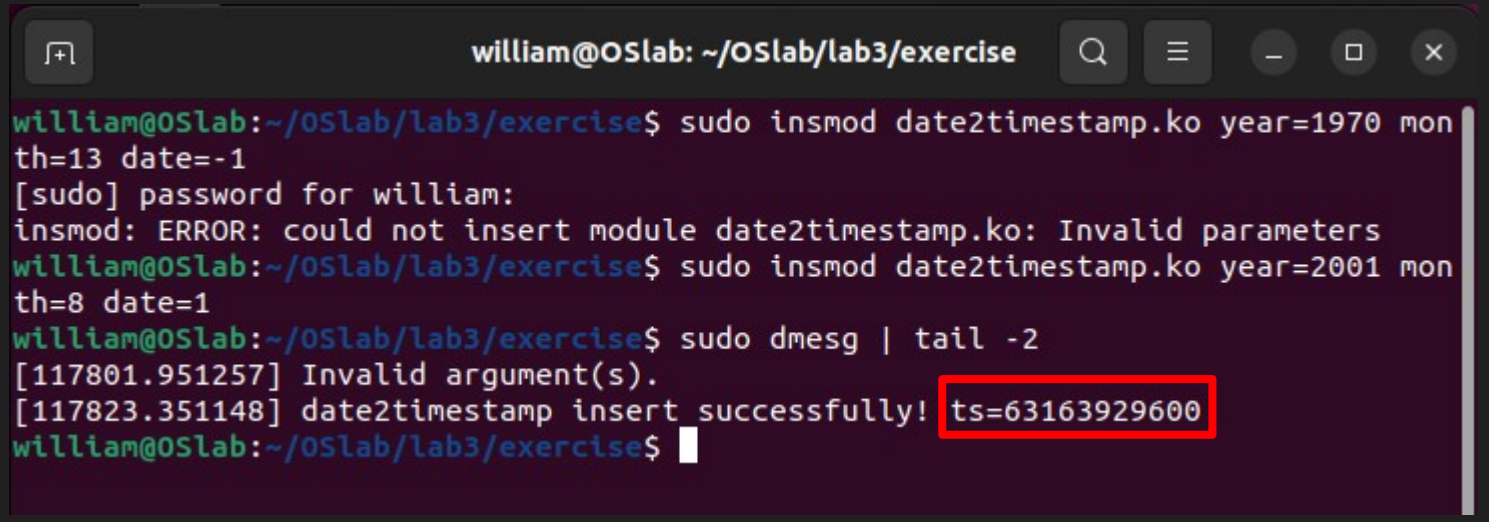
- 請撰寫兩個 **kernel module**
  - **date2timestamp**
    - **model parameters: year, month, date**
    - **export symbol: ts**
    - 輸入你的出生年月日**(y, m, d)**，印出該日期的時間戳記**(ts)** 並提供給其它模組使用
    - 若輸入無效日期，需顯示錯誤訊息並終止安裝

$ Exercise (2/5)

- 請撰寫兩個 **kernel module** (Cont.)
  - **myage**
    - 首先取得當前的時間戳記，使其跟 **date2timestamp** 的 **ts** 變數相減
    - 將算出來的時間戳記轉成年跟日，輸出你已經活多久了
- 統一寫在 **init_module** 函式內即可，可以不用額外建立函式或檔案
- **timestamp** 的精度為「秒」

$ Exercise (3/5)

- 範例輸出
  - 首先安裝 **date2timestamp**，若成功需印出 **ts** 的值，其餘文字不須照打，看得懂即可



$ Exercise (4/5)

- 範例輸出 (Cont.)
  - 再安裝 **myage** 輸出年跟日，若相減結果小於 **0** 需要提示

```
william@OSlab:~/OSlab/lab3/exercise$ sudo insmod myage.ko
william@OSlab:~/OSlab/lab3/exercise$ sudo dmesg | tail -2
[117903.157897] myage insert successfully!
[117903.157907] Wow you have survived on this world 23 years and 5 days!
```

$ Exercise (5/5)

- 提示：若使用者輸入無效日期，可執行以下兩行

```
printk(KERN_ERR "Invalid argument(s).\n");
return –EINVAL;   //  回傳負整數以中斷安裝
```

轉 timestamp 可以用

```
#include <linux/time64.h>
mktime64(year, month, date, 0, 0, 0);
```

- 現場 Demo　如下範例輸出即可，做不完也可以回家截圖並上傳至 **iLearning**
- **Due day: 2026/10/14 23:30**



Thanks for your listening

Made by William in 2024/08